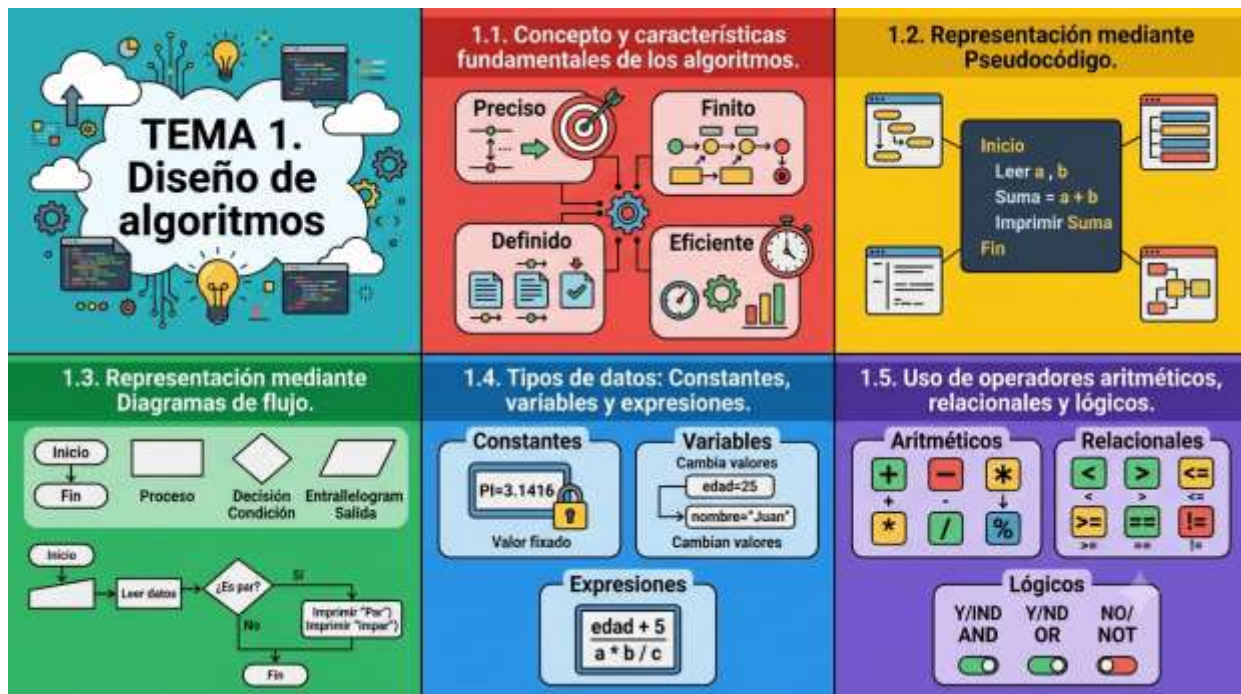


TEMA N° 1



TEMA 1 DISEÑO DE ALGORITMOS



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Comprender y estructurar la lógica algorítmica** necesaria para resolver problemas tecnológicos reales, aplicando un enfoque ordenado y sistemático que sirva como base sólida para tu formación como Técnico en Sistemas Informáticos.
2. **Dominar la representación de soluciones** mediante el uso hábil de pseudocódigo y diagramas de flujo estructurados, permitiéndote comunicar procesos informáticos de manera clara y estandarizada.
3. **Manipular tipos de datos, constantes, variables y operadores** lógicos, aritméticos y relacionales de manera óptima, garantizando el correcto procesamiento de la información en cualquier entorno de desarrollo de software.



PRÁCTICA



¿Qué tienen en común preparar un buen majadito chiquitano, organizar la salida de los trenes de carga o registrar las ventas diarias de un comercio? Si lo piensas detalladamente, verás que todas estas actividades requieren seguir una secuencia ordenada de pasos sencillos para alcanzar un resultado exitoso. Si alteras el orden de los pasos (por ejemplo, intentar servir el majadito antes de cocinar el arroz, o despachar un tren sin verificar las vías), el resultado final será un fracaso absoluto. En el mundo de la informática ocurre exactamente lo mismo: las computadoras no adivinan lo que queremos hacer; necesitan instrucciones ultra precisas y perfectamente ordenadas.

SITUACIÓN PROBLEMÁTICA REAL EN LA POBLACIÓN DE PAILÓN

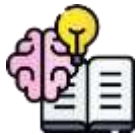
Imagina que la asociación de comerciantes del **Barrio Central Fátima**, en coordinación con la administración de la estación de trenes del **Barrio 24 de Septiembre**, ha decidido automatizar el control de recepción y distribución de mercancías pesadas (como bolsas de soya y maíz) que llegan a la población de Pailón. Actualmente, los camiones distribuidores sufren retrasos críticos porque el registro se realiza a mano en cuadernos, generando cuellos de botella que afectan el abastecimiento de los negocios locales y las tiendas en los barrios **Saó, María Belén y San Antonio**.

Como futuro Técnico en Sistemas Informáticos del **CEA Herman Ortiz Camargo**, se te ha encomendado la tarea de diseñar el flujo lógico para un nuevo módulo informático que se instalará en el flamante edificio educativo del CEA. Este módulo deberá:

1. Recibir la cantidad de toneladas de grano que trae un camión desde la estación ferroviaria.
2. Verificar si el peso excede la capacidad permitida para el ingreso al centro comercial del Barrio Central Fátima (máximo 12 toneladas por seguridad vial).
3. Calcular automáticamente el impuesto de descarga local (una tasa fija por tonelada).
4. Mostrar un reporte en pantalla indicando si el camión está "AUTORIZADO" o "RECHAZADO", junto con el monto total a pagar por el concepto de peaje.



Antes de sentarte a escribir código en un lenguaje de programación complejo como Java, Python o C++, necesitas estructurar la solución en papel. No pienses aún en comandos informáticos; piensa en la lógica pura: ¿Qué datos necesitas pedirle al operador? ¿Qué cálculos debes hacer primero? ¿Qué decisiones debe tomar el sistema según el peso del camión? Analizar este problema paso a paso es, precisamente, hacer un diseño de algoritmos.



TEORÍA



1.1. CONCEPTO Y CARACTERÍSTICAS FUNDAMENTALES DE LOS ALGORITMOS

Un algoritmo se define formalmente como un conjunto finito de pasos o instrucciones ordenadas, lógicas y bien definidas que permiten resolver un problema específico o realizar una tarea determinada. En el ámbito de la informática, un algoritmo es el plano de construcción o la receta que le indica a la computadora con absoluta precisión qué acciones debe ejecutar.

Para que una secuencia de pasos sea considerada un algoritmo válido en el desarrollo de sistemas, debe cumplir rigurosamente con las siguientes características fundamentales:

1. **Precisión:** Cada paso de la secuencia debe estar claramente definido, sin ambigüedades. No se permiten interpretaciones subjetivas. Si una instrucción dice "calcular el impuesto", debe especificar matemáticamente cómo hacerlo.
2. **Determinismo (Finitud de resultados):** Si se sigue el algoritmo dos o más veces utilizando los mismos datos de entrada, se debe obtener siempre el mismo resultado. Si el camión de soya pesa 10 toneladas hoy y 10 toneladas mañana, el impuesto calculado debe ser idéntico en ambos casos.
3. **Finitud:** El algoritmo debe tener un número limitado de pasos. Todo proceso debe terminar en algún momento; un algoritmo que se queda atrapado en un ciclo infinito no es útil y consume los recursos del hardware.
4. **Entrada (Input):** Corresponde a los datos iniciales que recibe el algoritmo para poder operar. Son la materia prima del proceso.



5. **Proceso:** Es la serie de operaciones, cálculos matemáticos y tomas de decisiones que transforman los datos de entrada en resultados útiles.
6. **Salida (Output):** Es el resultado final obtenido tras la ejecución del proceso. Puede ser un mensaje en pantalla, un ticket impreso o un registro guardado en una base de datos.

Como Técnico, debes comprender que el diseño de un algoritmo es completamente independiente del lenguaje de programación que utilices más adelante. La lógica es universal: si tu algoritmo está mal diseñado, fallará tanto si lo programas en PSeInt como si lo haces en la plataforma en la nube más avanzada.

1.2. REPRESENTACIÓN MEDIANTE PSEUDOCÓDIGO

El pseudocódigo (falso lenguaje) es una de las herramientas de diseño más potentes y utilizadas por los Técnicos en Sistemas Informáticos. Se trata de una forma de expresar los pasos de un algoritmo utilizando un lenguaje humano (en nuestro caso, castellano) estructurado con ciertas convenciones de la programación. No tiene la rigidez estricta de un compilador, pero mantiene un orden formal que facilita su posterior traducción a código real.

La principal ventaja del pseudocódigo es que permite al diseñador concentrarse en la lógica del problema en lugar de preocuparse por las reglas sintácticas complejas de un lenguaje informático (como los puntos y comas, las llaves o la gestión de memoria).

Para estandarizar nuestro aprendizaje en el primer semestre del CEA Herman Ortiz Camargo, utilizaremos la sintaxis clásica de la herramienta **PSeInt**. A continuación, se presenta la estructura básica de un pseudocódigo aplicada al problema del control de transporte de carga en Pailón:

```
Algoritmo ControlDeCargaPailon    // Línea 1: Se definen Las variables que
almacenarán Los datos numéricos del problema    Definir pesoToneladas,
tasaImpuesto, totalPagar Como Real    // Línea 2: Se define una variable de
texto para almacenar el estado de La autorización    Definir estadoVehiculo Como
Cadena    // Línea 3: Se asigna el valor de La tasa fija por tonelada (50
Bolivianos por tonelada)    tasaImpuesto <- 50.0    // Línea 4: Mostramos un
mensaje en La pantalla para solicitar datos al operador en el Barrio Central
Fátima    Escribir "
Ingrese el peso total del camión en toneladas:"    // Línea 5: El sistema
captura el dato introducido por el teclado y lo guarda en la variable
correspondiente    Leer pesoToneladas    // Línea 6: Se evalúa mediante una
estructura condicional si el peso supera el límite seguro de 12 toneladas    Si
```



```
pesoToneladas > 12.0 Entonces           // Línea 7: Si la condición es verdadera,
el camión excede el peso y se deniega su acceso      estadoVehiculo <- "
RECHAZADO por exceso de peso vial"           // Línea 8: A pesar del rechazo, se
calcula el impuesto correspondiente al pesaje obligatorio      totalPagar <-
pesoToneladas * tasaImpuesto      Sino           // Línea 9: Si la condición es
falsa, el camión cumple con los estándares viales de Pailón
estadoVehiculo <- "
AUTORIZADO para ingreso comercial"           // Línea 10: Se calcula el impuesto
regular de circulación interna      totalPagar <- pesoToneladas * tasaImpuesto
FinSi           // Línea 11: Se imprimen en la consola del sistema los resultados
finales para conocimiento del operador      Escribir "
--- REPORTE DE CONTROL VIAL PAILÓN -
--"      Escribir "
Estado del vehículo: ", estadoVehiculo      Escribir "
Total a cancelar de tasas municipales: ", totalPagar, " Bs."
FinAlgoritmo
```

Analiza detenidamente cómo cada instrucción tiene un propósito único, cómo los comentarios internos explican detalladamente la función de cada bloque, y cómo las sangrías (espacios al inicio de las líneas) ayudan a que el texto sea perfectamente legible para cualquier otro Técnico del equipo.

1.3. REPRESENTACIÓN MEDIANTE DIAGRAMAS DE FLUJO

Un diagrama de flujo es la representación gráfica de un algoritmo. Utiliza símbolos geométricos estandarizados por organismos internacionales (como ANSI e ISO) interconectados por flechas llamadas "líneas de flujo" que indican la dirección exacta en la que se ejecutan las instrucciones de la computadora.

Esta representación es sumamente didáctica, ya que permite apreciar de un solo vistazo la lógica del sistema, los caminos alternativos creados por las condiciones y el punto exacto de finalización del proceso. Los símbolos fundamentales que todo Técnico debe dominar e implementar son los siguientes:

1. **Óvalo o Elipse (Inicio / Fin):** Representa el punto de partida y de llegada del algoritmo. Ningún diagrama puede carecer de estos componentes.
2. **Paralelogramo (Entrada / Salida de datos):** Indica una operación de lectura por teclado (introducción de datos por parte del usuario) o una instrucción de escritura en pantalla (mostrar resultados o mensajes).



3. **Rectángulo (Proceso):** Representa cualquier operación interna realizada por la CPU, como asignaciones de valores, cálculos aritméticos, fórmulas matemáticas o manipulación de datos en memoria.
4. **Rombo (Decisión / Condición):** Indica un punto donde el flujo debe bifurcarse en función del resultado de una condición lógica o relacional. El rombo tiene una entrada y siempre cuenta con dos salidas etiquetadas como "Sí" (Verdadero) y "No" (Falso).



Reglas estrictas para el dibujo de diagramas de flujo en Systems Informatics:

1. Los diagramas deben dibujarse de arriba hacia abajo y de izquierda a derecha.
2. Las líneas de flujo deben ser completamente rectas; no se permite el cruce descuidado de líneas sin conectar.
3. Todo símbolo (a excepción del de Fin) debe tener una línea de salida conectada al siguiente paso.



1.4. TIPOS DE DATOS: CONSTANTES, VARIABLES Y EXPRESIONES

Para procesar información, una computadora necesita organizar la memoria volátil de forma eficiente. Aquí es donde entran en juego las variables y las constantes, que son esencialmente contenedores o casilleros con un nombre asignado dentro de la memoria RAM del sistema.

Tipos de Datos Primitivos

Antes de reservar un casillero, debemos indicarle a la computadora qué tipo de contenido vamos a almacenar en él. Los tipos fundamentales en la programación estructurada son:

1. **Entero (Integer):** Almacena números sin decimales, ya sean positivos o negativos. Ejemplo: El número de estudiantes inscritos en el módulo del CEA (por ejemplo, 35), o la cantidad de partidos jugados en el coliseo del **Barrio 27 de Mayo**.
2. **Real (Float / Double):** Almacena números que incluyen una parte decimal. Ejemplo: El precio de un pasaje en tren desde Pailón (15.50 Bs.) o los kilogramos exactos de soya de un cargamento.
3. **Carácter (Char):** Guarda un único símbolo, letra, dígito o signo de puntuación delimitado por comillas. Ejemplo: La sección de un aula ('A') o el tipo de categoría de transporte ('M').



4. **Cadena de Texto (String):** Almacena secuencias de caracteres, palabras completas o textos extensos. Ejemplo: El nombre completo del Técnico en Sistemas Informáticos ("Rubén Condori") o la dirección de un predio en el **Barrio Residencial Norte**.
5. **Lógico o Booleano (Boolean):** Solo puede tomar uno de dos valores posibles: Verdadero (True) o Falsos (False). Se utiliza ampliamente para banderas lógicas y estados del sistema. Ejemplo: tieneDeudas <- Falso.

Variables vs. Constantes

1. **Variable:** Es un espacio de memoria cuyo valor asignado **puede cambiar** a lo largo de la ejecución del algoritmo. Por ejemplo, la variable temperaturaPailon puede iniciar la mañana en 22.5 grados y subir a las tres de la tarde a 36.0 grados.
2. **Constante:** Es un espacio de memoria cuyo valor permanece **inamovible** e invariable durante toda la ejecución del programa. Se definen para proteger valores fijos del negocio o leyes de la naturaleza. Ejemplo: La constante del Impuesto al Valor Agregado (IVA <- 0.13) o el valor matemático Pi (PI <- 3.1416).

Expresiones

Una expresión es una combinación válida de variables, constantes, valores literales y operadores que la computadora evalúa para producir un nuevo resultado final. Por ejemplo: montoFinal <- subtotal - (subtotal * descuento).

1.5. USO DE OPERADORES ARITMÉTICOS, RELACIONALES Y LÓGICOS

Los operadores son los símbolos que le indican al procesador qué acción matemática, comparativa o lógica debe realizar sobre los datos alojados en las variables. Son las herramientas para construir expresiones útiles.

Operadores Aritméticos (Matemáticos)

Se utilizan para realizar cálculos numéricos tradicionales. El resultado de una expresión aritmética es siempre otro número.

Operador	Operación	Ejemplo	Resultado
+	Suma	15 + 5	20



-	Resta	100 - 35	65
*	Multiplicación	12 * 4	48
/	División	25 / 2	12.5
% o MOD	Residuo de la división entera	10 % 3	1

Operadores Relacionales (Comparativos)

Se emplean para evaluar la relación de igualdad o desigualdad entre dos valores del mismo tipo. El resultado de una expresión relacional es siempre un valor de tipo **Lógico** (Verdadero o Falso).

Operador	Significado	Ejemplo	Resultado
>	Mayor que	18 > 12	Verdadero
<	Menor que	5 < 2	Falso
>=	Mayor o igual que	12 >= 12	Verdadero
<=	Menor o igual que	7 <= 4	Falso
= o ==	Igual a	10 = 10	Verdadero
!= o <>	Distinto / Diferente de	5 != 5	Falso

Operadores Lógicos (Booleanos)

Permiten combinar múltiples expresiones relacionales simples para construir condiciones complejas y robustas dentro de un algoritmo. Operan únicamente con valores lógicos y devuelven un valor lógico.

1. **Operador Y (AND):** Devuelve Verdadero **únicamente si todas** las condiciones individuales evaluadas son verdaderas. Si falla una sola, el resultado completo se vuelve falso. Ejemplo: Para ingresar a un laboratorio del CEA se requiere que el alumno esteInscrito = Verdadero **Y** que tengaCredencial = Verdadero.
2. **Operador O (OR):** Devuelve Verdadero si **al menos una** de las condiciones evaluadas se cumple. Da un resultado falso solamente cuando todas las premisas son falsas. Ejemplo:



Un cliente del Barrio Central Fátima recibe un descuento si realiza su pago con Tarjeta = Verdadero **O** si es Día De Feria = Verdadero.

3. **Operador NO (NOT):** Es un operador de negación que invierte el estado lógico actual de la variable o expresión sobre la que actúa. Si la variable es verdadera, la transforma en falsa, y viceversa.

1.6. PENSAMIENTO ALGORÍTMICO Y MODULARIZACIÓN DE PROCESOS COMPLEJOS

Cuando un Técnico se enfrenta a retos de gran envergadura en entornos reales, intentar solucionar todo el problema de un solo golpe suele conducir a códigos engorrosos, difíciles de depurar y propensos a errores fatales. Aquí toma relevancia el pilar fundamental de la informática: "**Divide y vencerás**", conocido técnicamente como modularización o diseño descendente.

La modularización consiste en descomponer un problema complejo en subproblemas más pequeños, manejables e independientes. Cada uno de estos subproblemas se soluciona mediante un módulo o subproceso específico. En la realidad operativa de Pailón, consideremos la gestión completa de la estación ferroviaria en el **Barrio 24 de Septiembre**: controlar la llegada de los vagones, registrar el tipo de carga, calcular los salarios de los operarios de descarga y emitir los reportes tributarios para la alcaldía. Un solo algoritmo gigante sería inmanejable.

A través del pensamiento algorítmico avanzado, el Técnico fragmenta el sistema en cuatro bloques independientes:

1. Modulo_Control_Vagones
2. Modulo_Registro_Soya
3. Modulo_Planilla_Sueldos
4. Modulo_Reportes_Alcaldia

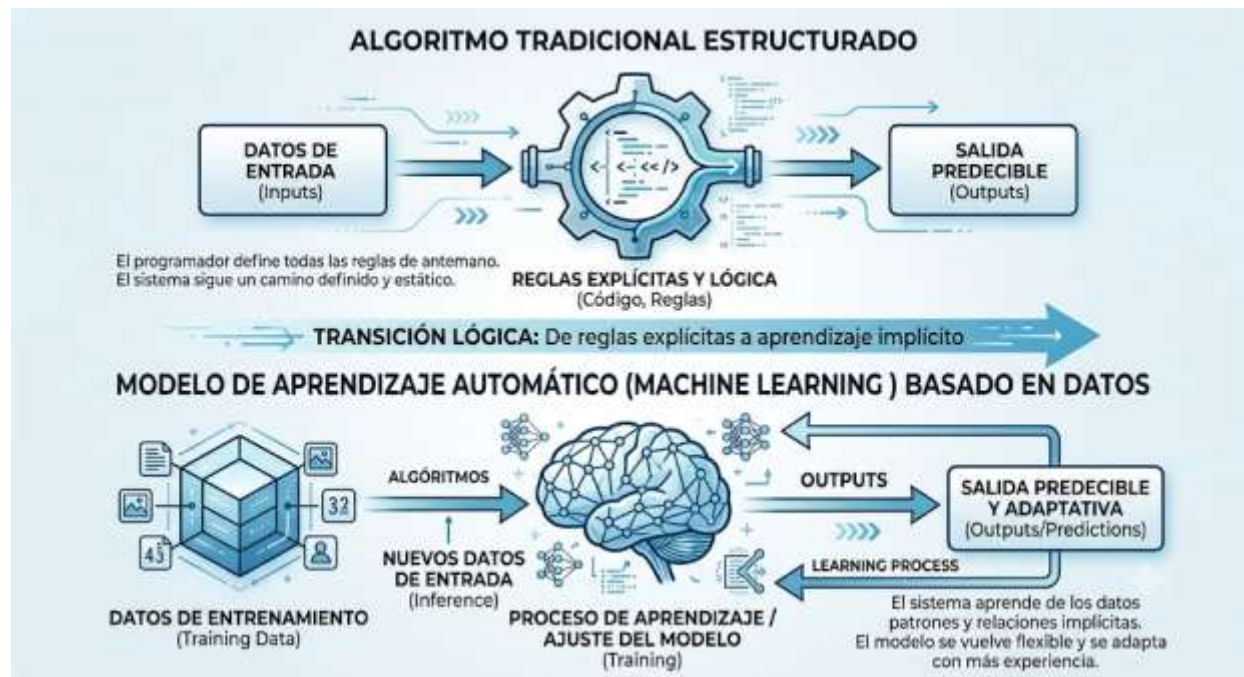
Esta abstracción no solo simplifica el desarrollo inicial dentro de las aulas del CEA, sino que permite que múltiples Técnicos trabajen simultáneamente en diferentes módulos sin pisarse los cables, facilitando un mantenimiento limpio a largo plazo y la reutilización de procesos enteros en futuros sistemas.



1.7. INTRODUCCIÓN AL DISEÑO DE ALGORITMOS AUTOMATIZADOS CON INTELIGENCIA ARTIFICIAL

Hoy en día, el rol del Técnico en Sistemas Informáticos está evolucionando a pasos agigantados gracias a la incorporación de la Inteligencia Artificial (IA). Los algoritmos tradicionales que estudiamos en este módulo son de naturaleza puramente determinista y estructurada: el programador debe prever manualmente todas las variables, condiciones y salidas posibles escribiendo reglas fijas paso por paso.

Sin embargo, los algoritmos modernos basados en Machine Learning (Aprendizaje Automático) operan de una manera radicalmente diferente. En lugar de recibir reglas fijas escritas por un humano, estos algoritmos son alimentados con millones de datos históricos de entrada y salida para que el propio sistema informático "aprenda" y descubra los patrones ocultos por sí mismo.



Por ejemplo, si alimentamos un algoritmo de IA con los registros históricos de consumo eléctrico de los últimos 5 años de todas las viviendas en los barrios **Oto Waver**, **Ovidio Barberi** y **San Expedito**, el sistema podrá generar de forma autónoma predicciones precisas sobre cuánta energía demandará la población de Pailón durante los meses de invierno o las festividades de fin de año, adaptando el software a entornos dinámicos y predictivos que van más allá de un simple cálculo matemático básico.



1.8. OPTIMIZACIÓN DE ALGORITMOS Y PREVENCIÓN DE BUCLES INFINITOS EN SISTEMAS DEL MUNDO REAL

Escribir un algoritmo que funcione es solo la mitad del trabajo de un Técnico competente; la otra mitad consiste en asegurarse de que el algoritmo sea eficiente y optimizado. La eficiencia de un proceso de software se mide principalmente en dos recursos críticos de la computadora: el tiempo de ejecución en la CPU y el consumo de espacio dentro de la memoria RAM.

Un error clásico y extremadamente peligroso que cometen los diseñadores principiantes es la creación involuntaria de un **bucle infinito**. Un bucle infinito ocurre cuando una estructura de repetición (como un ciclo Mientras o Para) carece de una condición de parada adecuada o una variable de control que incremente su valor de forma correcta. Como consecuencia, la computadora repite las mismas instrucciones eternamente, lo que satura el procesador al 100%, eleva drásticamente la temperatura del hardware y termina colgando por completo el sistema operativo de la empresa u organización.

Optimizar implica limpiar el código de pasos redundantes, evitar comparaciones innecesarias dentro de las bases de datos y verificar minuciosamente que cada estructura cíclica tenga una salida garantizada. Un software corporativo optimizado reduce costos de infraestructura en servidores, consume menos energía eléctrica en las terminales del CEA Herman Ortiz Camargo y garantiza una experiencia de usuario fluida, rápida y sumamente profesional.

CIERRE TEÓRICO OBLIGATORIO

❑ *Mitos vs. Realidades*

Mito	Realidad
Los algoritmos solo sirven para programar computadoras y crear aplicaciones complejas.	Los algoritmos son estructuras lógicas universales que aplicamos a diario para resolver problemas organizados, desde una receta de cocina hasta la logística de transporte en Pailón.
Para diseñar un algoritmo eficiente es obligatorio dominar un lenguaje de programación avanzado.	El algoritmo se diseña en papel, pseudocódigo o diagramas de flujo utilizando lógica pura. El lenguaje de programación es solo una herramienta posterior para traducir esa lógica.



□ *Glosario de Términos*

1. **Pseudocódigo:** Lenguaje intermedio de diseño que combina la estructura formal de la programación con la flexibilidad y legibilidad del idioma humano.
2. **Variable:** Espacio nombrado en la memoria RAM de una computadora cuyo contenido numérico o textual puede sufrir modificaciones durante la ejecución del programa.
3. **Bucle Infinito:** Fallo crítico de programación que se produce cuando una secuencia de instrucciones cíclicas se repite de manera indefinida por carecer de una condición de salida válida.
4. **Diagrama de Flujo:** Representación esquemática y visual de un algoritmo informático que emplea figuras geométricas conectadas por flechas directrices.
5. **Modularización:** Proceso arquitectónico de software que desglosa un problema informático complejo en pequeños componentes independientes denominados módulos.

□ *Ponte a Prueba*

1. **¿Cuál de las siguientes opciones define mejor la característica de "Finitud" en un algoritmo informático?**
 1. A) Que el algoritmo produzca siempre resultados numéricos decimales precisos.
 2. B) Que el algoritmo cuente obligatoriamente con un número limitado de pasos y termine en algún momento.
 3. C) Que el algoritmo pueda ejecutarse en múltiples sistemas operativos en la nube simultáneamente.
2. **Si declaramos la expresión lógica $(10 > 5)$ O $(3 = 7)$, ¿cuál será el resultado de la evaluación por parte del procesador?**
 1. A) Falso
 2. B) Verdadero
 3. C) Error sintáctico en memoria



3. ¿Qué símbolo geométrico se debe emplear en un diagrama de flujo para representar la lectura de una variable por teclado?

1. A) Un rectángulo perfectamente centrado.
2. B) Un paralelogramo con líneas diagonales limpias.
3. C) Un rombo con dos salidas lógicas opuestas.

Respuestas correctas para autoevaluación: 1=B, 2=B, 3=B.



VALORACIÓN



Como futuros profesionales y Técnicos en Sistemas Informáticos, el desarrollo de un algoritmo robusto no debe limitarse exclusivamente a cumplir con los requerimientos técnicos de la programación o generar ganancias económicas a una corporación; debe pasar obligatoriamente por un filtro de profunda responsabilidad ética, social y medioambiental.

En primer lugar, analicemos el fenómeno del **e-waste (basura electrónica)**. Un software mal diseñado, carente de optimización y con un consumo excesivo de ciclos de CPU y memoria RAM, obliga a los usuarios y comercios locales del **Barrio Central Fátima** a desechar prematuramente sus computadoras y servidores antiguos, creyendo erróneamente que el hardware ya no sirve, cuando en realidad la culpa es del código ineficiente del programador. Esto acelera de forma alarmante la acumulación de desechos tecnológicos altamente contaminantes en los vertederos del departamento de Santa Cruz, liberando metales pesados al suelo y dañando nuestro entorno natural chiquitano.

+-----+|

IMPACTO DE LA OPTIMIZACIÓN

|+-----+

|Código Ineficiente -> Alto consumo CPU -> Desecho de Equipos|

| Código Optimizado -> Menor uso hardware-> Vida útil prolongada|+-----+

En segundo lugar, la **seguridad y la privacidad de los datos** de los ciudadanos de la población de Pailón deben ser la prioridad absoluta al diseñar la lógica de un sistema. Cuando estructures un algoritmo de almacenamiento o recolección de datos en barrios como **Las Misiones, San**



José o Jardines de Pailón, debes implementar validaciones estrictas y flujos de control que impidan el acceso no autorizado a información sensible de los usuarios. Un descuido algorítmico puede dejar expuesta la base de datos de cobranzas o registros médicos a ataques cibernéticos, destruyendo por completo la confianza de la comunidad en los avances tecnológicos y afectando directamente la estabilidad social del municipio. Tu firma como Técnico debe ser sinónimo de integridad, eficiencia y resguardo del bien común.



PRODUCCIÓN



RETO FINAL: SISTEMA INTEGRADO DE CONTROL DE TAQUILLA PARA EL COLISEO MUNICIPAL DEL BARRIO 27 DE MAYO

Objetivo: Desarrollar un algoritmo integral en pseudocódigo (compatible con PSeInt) que automatice la venta de entradas para los campeonatos interbarriales de fútbol de salón celebrados en el Coliseo del **Barrio 27 de Mayo**, aplicando de forma estricta constantes, variables, operadores aritméticos, relacionales y lógicos combinados.

Requerimientos del Sistema:

1. **Constantes Obligatorias:** El precio base de la entrada general al coliseo es una constante fija de 30.0 Bs. La capacidad máxima de seguridad permitida para el evento es de 500 espectadores.
2. **Entradas por Teclado:** El operador del sistema debe registrar el nombre del asistente, su edad y la cantidad de entradas que desea adquirir.
3. **Lógica del Descuento:** * Si el asistente reside en el **Barrio Saó** (donde se encuentra nuestro CEA) **Y** su edad es menor o igual a 12 años, se le aplica de forma automática un descuento del 50% sobre el valor total de sus entradas por incentivo deportivo infantil.
 1. En cualquier otro caso, el asistente cancela la tarifa general base completa sin rebajas.



4. **Control de Capacidad:** El sistema debe verificar si las entradas solicitadas no superan la capacidad disponible del coliseo. De ser viable, aprueba la transacción; de lo contrario, cancela la venta emitiendo una alerta.

Guía Paso a Paso para la Computadora:

```

Algoritmo ControlTaquillaColiseo27DeMayo    // Paso 1: Definición explícita de
todas las variables con su tipo correspondiente    Definir nombreAsistente,
barrioOrigen Como Cadena    Definir edadAsistente, cantidadEntradas,
capacidadMaxima Como Entero    Definir PRECIO_BASE, costoSubtotal,
montoDescuento, montoTotalAPagar Como Real    Definir ventaAprobada Como Logico
// Paso 2: Inicialización estricta de las constantes y variables de control del
coliseo    PRECIO_BASE <- 30.0    capacidadMaxima <- 500    montoDescuento <-
0.0    ventaAprobada <- Falso    // Paso 3: Captura de datos iniciales del
usuario por medio de la consola del sistema    Escribir "=== SISTEMA DE TAQUILLA
- COLISEO BARRIO 27 DE MAYO ==="    Escribir "
Ingrese el nombre completo del asistente:"    Leer nombreAsistente
Escribir "
Ingrese el barrio de procedencia del cliente:"    Leer barrioOrigen
Escribir "
Ingrese la edad actual del asistente:"    Leer edadAsistente    Escribir "
Ingrese la cantidad de entradas que desea comprar:"    Leer cantidadEntradas
// Paso 4: Evaluación lógica condicional combinada para la aprobación de la
capacidad vial del evento    Si cantidadEntradas > 0 Y cantidadEntradas <=
capacidadMaxima Entonces    ventaAprobada <- Verdadero
capacidadMaxima <- capacidadMaxima - cantidadEntradas // Actualización de los
cupos disponibles en el coliseo    Sino    ventaAprobada <- Falso
Escribir "
ERROR CRÍTICO: La cantidad solicitada es inválida o supera el aforo permitido
del coliseo."    FinSi    // Paso 5: Cálculo matemático financiero en caso
de que la transacción haya sido aprobada lógicamente    Si ventaAprobada =
Verdadero Entonces    costoSubtotal <- cantidadEntradas * PRECIO_BASE
// Estructura selectiva para aplicar el beneficio por ser estudiante vecino del
Barrio Saó en la infancia    Si (barrioOrigen = "
Barrio Saó" O barrioOrigen = "
Saó") Y edadAsistente <= 12 Entonces    montoDescuento <- costoSubtotal
* 0.50    Escribir "|BENEFICIO APLICADO!: Se ha otorgado un 50% de
descuento por incentivo vecinal infantil del Barrio Saó."    Sino
montoDescuento <- 0.0    FinSi    montoTotalAPagar <-
costoSubtotal - montoDescuento    // Paso 6: Despliegue de
resultados y emisión del ticket digital formal para el control de acceso
Escribir ""    Escribir "
---
---
---
---
---

```




```
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
--"      Escribir "¡Disfrute del campeonato en el Coliseo Municipal de  
Pailón!"  FinSiFinAlgoritmo
```

RECURSOS ADICIONALES

1. **Documentación Oficial de PSeInt:** Accede al portal oficial de la herramienta educativa de programación para revisar los manuales de uso técnico, ejemplos prácticos de diagramación automática de flujo y la guía completa de configuración del lenguaje estructurado en español. URL: <http://pseint.sourceforge.net/>
2. **Repositorio Comunitario GitHub - Algoritmia Básica:** Explora proyectos y plantillas de lógica algorítmica estructurada diseñados por comunidades globales de desarrollo de software para profundizar tu dominio de las estructuras de control condicionales. URL: <https://github.com/topics/algorithms>
3. **Manual de Lógica de Programación - Fundamentos de Software:** Enlace didáctico de referencia académica que desglosa detalladamente la gestión avanzada de la memoria RAM de los dispositivos, el correcto nombrado de variables del sistema y las mejores prácticas internacionales de optimización de código. URL: https://developer.mozilla.org/es/docs/Learn/JavaScript/First_steps/Variables